

Code Guru: An Integrated Real-Time Learning Support Platform for Novice Java Programmers

W. P. R. Nethmina

*Dept. of Information Technology
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22253958@my.sliit.lk*

J. Aron Charles

*Dept. of Information Technology
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22203380@my.sliit.lk*

H. A. S. I. Madurapperuma

*Dept. of Information Technology
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22230942@my.sliit.lk*

Suriyaa Kumari

*Dept. of Information Technology
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
suriyaa.k@sliit.lk*

M. N. H. Appuhami

*Dept. of Information Technology
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22140852@my.sliit.lk*

Uthpala Samarakoon

*Dept. of Information Technology
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
uthpala.s@sliit.lk*

Abstract— Introductory programming courses report withdrawal and failure rates of 30–50%, and the tools novices rely on do little to change this: IDEs report errors without teaching, while AI assistants bypass the learning they should protect. This paper presents Code Guru, a platform that keeps novice Java programmers inside their normal coding environment while surrounding them with pedagogically constrained help, via four cooperating services: Code Coach, a VS Code extension detecting fifteen beginner error types with three-level scaffolded hints instead of fixes; Study Guider, an adaptive remediation engine converting repeated struggle into graph-RAG micro-lessons; PairPath, a behaviour-aware pair-programming classifier over five collaboration states; and an adaptive gamification engine. Three components are implemented and evaluated. Code Coach, trained on a 2,010-file verified synthetic corpus and tested on a hand-written holdout, reaches per-type F1 between 0.973 and 1.0, with candidate-level scoring raising per-site precision from 0.500 to 1.000 at under 4 ms per request. PairPath’s collaboration classifier and Study Guider’s struggle classifier and micro-lesson pipeline reach 0.879 and 0.91 macro-F1 respectively on synthetic and pilot data. We describe the shared architecture connecting all four components and the platform-level evaluation planned as future work.

Keywords— *programming education, intelligent tutoring systems, static analysis, machine learning, pair programming, adaptive remediation*

I. INTRODUCTION

Learning to program is a difficult transition, and the difficulty is not evenly distributed: first-year students and those arriving from non-technical backgrounds carry most of it. Studies of introductory courses report withdrawal and failure rates between 30% and 50%, driven less by syntax than by logical mistakes the student cannot see, error messages they cannot interpret, and repeated failure without guidance on what to do differently. In Sri Lanka, where workforce digitalization is a national priority, the cost of these early exits is felt beyond the classroom.

The tooling available to beginners does not address this problem well. Compilers and IDE linters detect what is mechanically wrong but explain nothing; conversely, the current generation of AI coding assistants resolves the task for the student, which removes exactly the productive struggle a novice needs [1], [2]. Research prototypes address individual slices of the problem, including compilation error

repair [1], reference-based feedback generation [3], novice-oriented static analysis [4], and behavioral monitoring of student coding [5], but these capabilities remain scattered across separate systems that were never designed to inform one another.

This research addresses those limitations directly: four components observe the same learner from different angles and share what they see. Code Coach watches the code as it is typed and turns mistakes into scaffolded hints; Study Guider watches the pattern of mistakes over time and intervenes with targeted micro-lessons; PairPath watches how two students collaborate and nudges an unproductive pair back into balance; the gamification engine watches demonstrated mastery and shapes practice accordingly. All four converse through one event vocabulary, so a struggle detected in the editor can become a lesson, a pairing prompt, or an adjusted practice plan without any component knowing the internals of another.

The paper makes three contributions. First, it describes the platform architecture: independently deployable services sharing a single identity provider and a typed learning-event contract. Second, it reports the design and full evaluation of Code Coach, including a detection principle we found essential, namely that a machine-learned gate belongs only in front of detection rules that must guess, and a candidate-level refinement of that principle that removes an entire class of false positives. Third, it summarizes the designs of the remaining components and the evaluation protocol prepared for the integrated platform.

II. RELATED WORK

Automated support for novice programmers has been studied from three directions relevant here. The first is error detection and repair. Ahmed et al. learn compilation-error fixes from student programs themselves [1], and Singh et al. generate feedback for introductory assignments by comparing submissions against reference implementations [3]; both produce corrections rather than guidance, and the latter requires an assignment-specific reference. Blok and Fehnker adapt static analysis to novice code and argue for pedagogically motivated rules [4], while Hassan et al. evaluate mainstream analyzers against engineering criteria that have little to do with learning [6]. Neural approaches locate probable defect sites but explain them weakly [7].

Commercial-style assistants such as AIRA [8] and CodeBlizz [9] integrate detection and AI suggestions into the workflow, but their orientation is developer productivity; Denny et al. show that students themselves prefer immediate help that stops short of revealing solutions [2], and CodeDive demonstrates real-time observation of student coding activity without pedagogical response [5].

The second direction is adaptive instruction. Intelligent tutoring systems have a long record of effectiveness reviews [10], and adaptive learning systems improve outcomes in programming specifically [11], [12]. Recent work builds the knowledge structures such systems need: knowledge-component graphs extracted from educational content [13], knowledge tracing tuned to programming exercises [14], and retrieval-augmented generation grounded in graphs rather than flat document stores [15]. Study Guider draws on this line, combining struggle classification with graph-grounded lesson generation and post-lesson verification.

The third direction is collaborative learning. Pair programming has documented benefits for confidence and retention in first courses [16], but its classroom model assumes an abler partner, and studies of novice-to-novice pairs report driver dominance, disengaged navigators, and unproductive dialogue patterns [17], [18]. Existing responses either analyse collaboration after the fact [17], restructure the activity statically [19], or replace the partner with an AI [20], which sacrifices the social dimension the practice exists to exploit. PairPath instead observes the pair's behavioral event stream live and intervenes without supplying solutions.

What distinguishes Code Guru from each line individually is the connection between them: detection, remediation, collaboration support, and motivation operate on shared signals about the same learner. To our knowledge no prior system for novice programmers closes that loop across all four concerns inside the student's own editor and study environment.

III. PLATFORM ARCHITECTURE

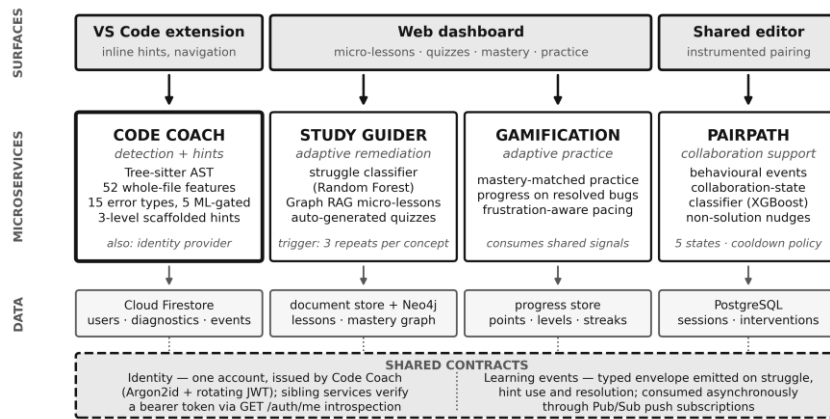


Fig. 1. Code Guru platform architecture.

IV. CODE COACH: REAL-TIME ERROR DETECTION

A. Detection pipeline

Code Coach is a TypeScript VS Code extension backed by a FastAPI service. The extension debounces editor changes (900 ms) and submits the current file; the backend parses it with Tree-sitter, extracts features, runs detection, and returns structured diagnostics that the extension renders as native

Fig. 1 shows the overall platform architecture. Code Guru is organized as four independently deployable micro-services, one per component, each owning its data store. The services communicate in two ways. Synchronous reads use REST over HTTPS: every read a downstream component needs is exposed by the owning service under a self-describing path (for example, /students/me/struggling concepts). Asynchronous reactions use published events: when Code Coach's analysis produces a remediation trigger, it emits a typed event that Study Guider consumes without Code Coach knowing who is listening. The event envelope carries a unique identifier, a version, a timestamp, and a typed payload, so consumers can deduplicate and evolve independently.

Identity is centralized without a separate identity service. Code Coach's backend, the first service built, owns registration and sign-in: passwords are hashed with Argon2id, sessions are represented by rotating refresh tokens with server-side revocation, and access tokens are short-lived JWTs. Sibling services validate a request by forwarding its bearer token to the owning service's introspection endpoint, which returns the verified user profile or rejects the token. Because tokens are bound to revocable server-side sessions, sign-out takes effect platform-wide immediately, which a shared-secret scheme could not guarantee. Credential endpoints are rate-limited per client address to resist brute-force attempts.

The reference deployment packages each service as a container. Code Coach's image (Python 3.12, 527 MB) runs on Google Cloud Run, which terminates TLS, load-balances, and scales to zero between sessions; its persistence is Cloud Firestore accessed through a repository seam that also supports MongoDB and an in-memory store for testing. The design deliberately avoids an API gateway and a dedicated load balancer: at platform scale, the managed runtime already provides both, and application-level JWT auth performs the gatekeeping.

squiggles, hover cards, an issue-navigation panel, and CodeLens actions. Each diagnostic carries an error type, position, confidence, a concept tag, and three hint levels authored per error type: a concept hint explaining the underlying idea, a guidance hint directing attention, and a targeted hint identifying what to reconsider. Corrected code is never shown, following the learner-agency findings in [2].

Every hint view, navigation step, resolution, and student-reported false positive is recorded as a typed learning event; the false-positive control doubles as a labeled data source for measuring real-world precision. Fig. 2 shows the resulting

output in the editor: the flagged loop condition, the hover card reporting the detection engine and confidence, and the panel presenting the three hint levels.

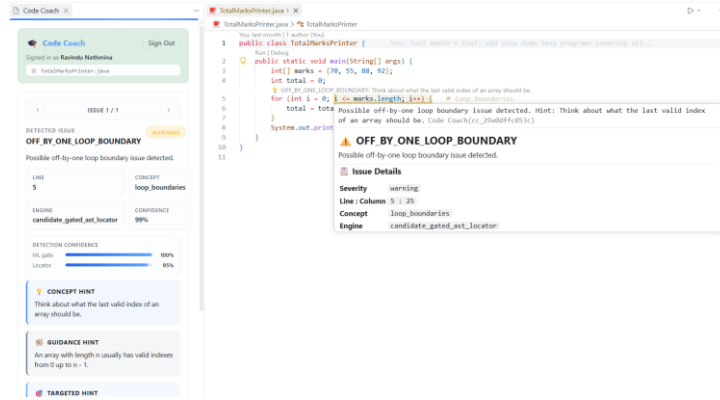


Fig. 2. Code Coach output in the editor.

B. Two-tier detection

The system detects fifteen beginner error types, registered in a single catalog that assigns each a detection mode. Ten types are found by AST rules that constitute proofs: a self-assignment or a division by a literal zero, once matched structurally, admits no innocent reading, so the rule runs ungated. Five types are different in kind, because their rules must be guessed. A switch case without a break may be intentional fall-through; a while condition whose variable is never textually updated may still terminate through side effects; a loop bound of $i \leq a.length$ is a defect, while $i \leq a.length - 1$ is correct Java that matches the same textual pattern. For these five, a per-type logistic-regression classifier estimates the probability that the file truly contains the defect, and the rule runs only above a calibrated threshold. The governing principle is that a gate belongs only where the rule guesses; placing one in front of a proof can only suppress true positives.

C. Verified synthetic training data

Hand-labeling enough Java to train five binary classifiers was impractical, and an early corpus of small uniform snippets produced a measurable failure discussed in Section VI. We instead generate training data. A parameterized generator produces minimal pairs, a buggy variant and its fix differing only in the defect, embedded at random positions in classes of one to nine methods, together with intentional negatives, correct code on which the crude rule still fires, such as commented fall-throughs and length-minus-one bounds. Every generated file is verified at generation time: it must parse completely, and the set of rule findings must equal the planted set exactly, so labels are correct by construction. The corpus used here contains 2,010 files from a fixed seed. Crucially, a separate corpus of 210 hand-written snippets is excluded from training and validation entirely and used only for testing, so reported test scores answer whether models trained on synthetic code transfer to code written by people.

D. Training and calibration

For each of the five gated types we train logistic regression, random forest, and RBF-SVM classifiers on 52 whole-file features (1,397 training rows, class-weighted), select by validation F1 with ties broken by inference latency, and calibrate the decision threshold per type: at the midpoint of

the validation separation margin when the classes separate, or by an F1 sweep when they overlap. Logistic regression was selected for all five types, which keeps the deployed models interpretable; each probability is the sigmoid of a weighted feature sum, and we verified the deployed implementation against a hand computation to ten decimal places.

E. Candidate-level gating

File-level gating has a structural blind spot that we predicted, demonstrated, and then removed. Whole-file features can tell the model that one loop in a file is genuinely defective and another merely resembles a defect, but aggregation discards which is which; when the gate opens, the crude rule flags both. Our fix changes the unit of analysis. For the off-by-one type, an extractor describes every for-loop with sixteen features local to that loop alone, the decisive one being whether the right side of a $\leq$ comparison is a bare `.length` field access or an arithmetic expression, a distinction visible in the syntax tree but not in text patterns. Labels for candidate training required no new data: the corpus generator already fixes each defect's location, so every loop in the existing 2,220 files could be labeled mechanically (1,594 candidates, 198 positive, 41 negatives on which the rule fires). At runtime the rule proposes every match and the candidate model disposes of each individually. Detection and localization merge, and sensitivity to file shape disappears by construction because no feature describes anything beyond the loop itself.

V. ADAPTIVE SUPPORT COMPONENTS

A. Study Guider: adaptive remediation

Hints are the right first response to a mistake, but a student who makes the same class of mistakes repeatedly needs more than a better hint. Study Guider consumes the learning events and struggle summaries that Code Coach exposes, classifies persistent struggle with a random-forest model over error repetition and attempt patterns, and treats three consecutive failures on one logical concept as an intervention trigger. The intervention begins as a notification card inside the IDE and continues on a web dashboard, where a graph-based retrieval-augmented generation pipeline composes a micro-lesson specific to the student's error in context, grounded in a curated concept corpus rather than free generation [15]. Each lesson

is followed by an automatically generated quiz that verifies understanding before the struggle is considered addressed, and mastery is maintained in a skill knowledge graph (Neo4j) that drives progress visualizations. The design responds directly to the knowledge-tracing and adaptive-learning literature [11]–[14]: remediation is triggered by evidence, targeted at a concept, and validated afterwards.

B. PairPath: behaviour-aware pair programming

Pair programming helps novices when the pairing works, but two novices cannot reliably recognize a stalled collaboration from inside it [16], [17]. PairPath instruments a shared editor so that every edit, execution, discussion note, and role switch becomes a timestamped behavioral event. A supervised gradient-boosted classifier consumes sliding-window features of this stream and assigns one of five collaboration states: productive, driver dominance, passive navigator, logic struggle, or disengaged. The classifier never inspects program correctness; it models how the pair is working, not whether the code compiles. When a non-productive state is detected above a confidence threshold, a rule-governed intervention layer responds with role-switch prompts, engagement nudges, or scaffolded concept hints drawn from a tagged corpus, under an explicit non-solution-giving constraint and a cooldown policy that prevents nagging. The architecture separates a real-time collaboration service (PostgreSQL, Socket.IO) from the ML inference service, mirroring the platform's general pattern of independently deployable concerns.

C. Adaptive gamification

The fourth component addresses motivation. It consumes the same mastery and struggle signals to adapt game-based practice, selecting exercises at the edge of demonstrated ability, awarding progress for resolved diagnostics rather than raw activity, and moderating challenge for students whose event history shows frustration. Its recommendation and session-result interfaces are already part of the platform's API surface; its full design and evaluation are reported separately.

VI. EVALUATION

The results in this section cover the three components with completed, measurable evaluations — Code Coach, PairPath, and Study Guider; adaptive gamification and platform-level, user-facing evaluation are discussed as future work at the end.

A. Detection performance

TABLE I. GATED DETECTION PER ERROR TYPE (LOGISTIC REGRESSION SELECTED FOR ALL FIVE)

Error type	Thresh.	Margin	Val F1	Test F1
Off-by-one loop boundary	0.530	0.94	1.00	0.973
Incorrect conditional operator	0.501	0.99	1.00	0.982
Array length index misuse	0.504	0.99	1.00	0.973
Missing break in switch	0.520	0.95	1.00	1.000
While variable not updated	0.503	0.99	1.00	1.000

Test F1 for the first three types is measured on the hand-written holdout (57 positives each); the two remaining types, added after the manual corpus was frozen, are tested on held-out synthetic units (27 positives each). Validation Brier

scores are 0.000 for all five types, indicating the probabilities are already well calibrated; post-hoc calibration was evaluated and found to have nothing to correct.

B. The gate encodes intent

The purpose of gating is judgment the rule lacks, and this is directly observable. An unintentional missing break receives gate probability 0.996 and is flagged; an intentional fall-through marked by a comment, on which the rule fires identically, receives 0.024 and is suppressed. The learned weights explain the behavior: the structural bare-length feature carries weight +2.74, roughly four times the crude textual patterns (+0.67), over an intercept of −6.69 that presumes innocence until evidence accumulates.

C. Out-of-distribution failure and repair

Models trained on an early corpus of small single-method snippets assigned the same defective loop probability 0.994 in a minimal file but 0.033 in a realistic five-method file: the whole-file context features had drifted outside the training distribution, and the gate suppressed a real defect. Retraining on the size-diverse generated corpus resolved this; the identical experiment now yields 0.999 in both files. We report this because the failure mode seems likely to affect any whole-file formulation trained on small examples, and because it motivated the candidate-level design, which is immune by construction.

D. File-level versus candidate-level gating

TABLE II. PER-SITE DETECTION UNDER THE TWO GATING ARCHITECTURES

Evaluation set	File-level	Candidate-level
Held-out test split (F1)	0.991	0.991
Mixed challenge, precision	0.500	1.000
Mixed challenge, recall	1.000	1.000

On ordinary held-out files the two architectures are indistinguishable, so candidate scoring costs nothing. The mixed challenge set contains fifty files, each holding one genuine off-by-one and one correct length-minus-one loop; the crude rule fires on both loops in all fifty (verified), so the set isolates gating behavior. File-level gating flags the innocent loop in every file, halving precision. Candidate-level gating accepts exactly the defective loop in all fifty. Per-candidate inference costs 0.006 ms, and end-to-end analysis, parsing through hint attachment, takes 3.9 ms at the median against a 900 ms debounce, so detection latency is imperceptible to the student.

E. PairPath: collaboration-state classification

PairPath's five-state classifier was evaluated on a synthetic corpus of 200 pair-programming sessions (40 per state, 12 simulated minutes each) generated from a seeded behavioural model in which inter-edit gap, discussion frequency, navigator-participation probability, run cadence, and role-rotation schedule are randomised per session around a per-state regime, so the planted state is correct by construction and never derived from classifier output. Sessions were sliced into 180-second windows and partitioned by session into train (120), validation (40), and test (40), so no session appears in more than one split. A grid search over five XGBoost hyperparameters, selected on validation macro-F1, replaced

an originally hardcoded configuration; the tuned model was refit on train+validation and scored once on the held-out test set.

TABLE III. PAIRPATH PER-STATE TEST RESULTS (603 WINDOWS, 40 SESSIONS)

State	Prec.	Rec.	F1	Supp.
Productive	0.881	0.872	0.876	187
Driver dominance	0.647	0.733	0.688	45
Passive navigator	0.893	0.934	0.913	152
Logic struggle	0.948	0.886	0.916	166
Disengaged	1.000	1.000	1.000	53
Accuracy / Macro F1			0.892 / 0.879	603

The classifier reaches 0.892 accuracy and 0.879 macro-F1 (Table III), improving on an expert rule-based baseline built from the same state definitions by +0.277 accuracy and +0.261 macro-F1. Disengaged separates cleanly (F1 1.000); driver dominance is the weakest state (F1 0.688), confused mainly with passive navigator, the one pair of states that differ only in role-rotation timing once edit share is excluded from both classifier and baseline by design, since the platform makes the navigator’s editor read-only. Feature extraction and prediction together cost a median 1.45 ms per window, well within budget for live intervention.

These figures measure how separable the generator’s planted states are, not classifier accuracy on real students: the labels are correct relative to design-time behavioural definitions unvalidated against observed human collaboration, and disengaged’s perfect score is plausibly an artefact of synthetic cleanliness. Test windows are also strongly correlated within a session, so the effective test size is closer to 40 than 603, leaving driver dominance’s precision wide-intervalled on only 45 windows. A mid-work revision to that definition, redefining it around role-rotation timing rather than navigator silence, raised macro-F1 from 0.834 to 0.879 — a gain from a clearer definition, not a stronger model.

F. Study Guider: struggle classification and micro-lesson pilot

Study Guider’s random-forest struggle classifier was evaluated on 200 synthetic attempt sequences, each mapping error-repetition, AST complexity, and past performance to a cognitive-state label planted by the generator. An 80/20 train/test split yields 0.92 macro-precision, 0.90 macro-recall, and 0.91 macro-F1, with a false-trigger rate of 0.08 (Table IV). This labelling is likewise synthetic and verified-by-construction rather than drawn from observed student logs, so real-world false-trigger rates are likely higher, mirroring the caveat attached to PairPath’s evaluation above.

TABLE IV. STUDY GUIDER EVALUATION RESULTS

Component	Metric	Value
A. Struggle classifier	Precision (macro)	0.92
	Recall (macro)	0.90
	F1 (macro)	0.91
	False-trigger rate	0.08
B. RAG micro-lessons (N=15 pilot)	Relevance to bug (1–5)	4.6
	Syllabus grounding (1–5)	4.8

The micro-lesson generator was assessed separately, since no automated grounding metric was available in the time available: three team members rated 15 dynamically generated lessons on a 1–5 scale for relevance to the flagged bug and for syllabus grounding, i.e. absence of hallucination against the retrieved concept context (Table IV). This is a small internal pilot rated by the system’s own developers rather than independent judges, which the evaluators’ own methodology notes as a likely source of confirmation bias; it demonstrates that the graph-RAG pipeline is architecturally viable, not that lesson quality holds at scale or under independent judgment. Mastery verification — whether a lesson and its follow-up quiz measurably reduce repeat errors on a concept — is not evaluated here, since establishing that requires longitudinal data from real students; we list it as future work in Section VIII.

VII. DISCUSSION AND LIMITATIONS

These results carry practical weight beyond the metrics themselves. File-level gating’s precision of 0.500 on the mixed challenge set means a novice would see a false alarm roughly every other time the off-by-one detector fires on adversarially shaped code — indistinguishable from chance. For a tool meant to protect productive struggle, not add noise to it [2], that erodes exactly the trust the platform depends on: candidate-level gating is not an incremental metrics gain but a correction to a design-critical flaw whole-file judgment could not see.

This focus also distinguishes Code Coach’s evaluation from adjacent tools: AIRA [8] and CodeBlizz [9] integrate detection into the developer workflow without reporting per-site precision, and Hassan et al. [6] evaluate analyzers against coverage and scalability rather than cases built to expose gating blind spots. Precision measured on a set engineered to defeat file-level gating, rather than an ordinary split where the two architectures are indistinguishable (Table II), isolates a failure mode an average-case benchmark would hide.

The 0.000 Brier score for all five gated types (Table I) is unusually clean but plausible: each gate is a single logistic sigmoid over a small, hand-designed feature set, bounding how miscalibrated its output can become relative to a high-capacity model, and the validation separation margins are large (0.94–0.99), meaning the classes are close to linearly separable. Post-hoc calibration corrects probabilities where a boundary is confidently wrong, and a boundary this clean leaves little to fix.

A near-perfect F1 on a 27–57-positive holdout invites a fair question: generalization, or an easy test? Two choices answer it directly. Training labels are correct by construction — a generated file is accepted only if its rule findings exactly match the planted defect (Section IV-C) — removing label noise as a confound; and the primary test set is hand-written by people, not generated by the training process, so a high score there answers whether synthetic-trained models transfer to code no generator produced. The two types tested only on held-out synthetic units (Table I) carry this claim more weakly, and we mark that distinction rather than treat all five results as equivalent evidence.

Six limitations bound the present results. First, metrics for all three measured components rely predominantly on synthetic, verified-by-construction data rather than real

student or pair behaviour; Code Coach’s hand-written holdout (Section IV-C) is the one direct test against unseen human-written artifacts, and the planned user study (Section VI-G) is intended to extend that test to PairPath and Study Guider. Second, PairPath’s per-state figures should be read at session rather than window level, since windows within a session are correlated (effective test n nearer 40 than 603), and its weakest state, driver dominance, rests on only 45 test windows. Third, Study Guider’s micro-lesson pilot (N=15) was rated by the development team itself, a likely source of confirmation bias per the team’s own report, and its mastery-verification stage has no evaluation at all, since demonstrating reduced repeat errors after an intervention requires longitudinal real-student data; together these show architectural viability, not independent, at-scale pedagogical benefit. Fourth, candidate-level gating is implemented for one Code Coach error type; extending it to the remaining four is mechanical but undone, and two loop defects outside the crude rule’s textual coverage are missed by both architectures — a rule-coverage gap rather than a gating gap. Fifth, all three measured components reason over a single student, pair, or file at a time; none yet models patterns across a cohort of learners. Sixth, Code Coach supports Java alone, tying its pipeline to a specific grammar, feature extractors, and locators; PairPath and Study Guider are comparatively language-agnostic by design but untested outside this setting.

Two design decisions deserve defense. Detection fails safe but never silent: if gating errors occur at runtime the system emits no gated diagnostics rather than unfiltered rule output, and logs the failure. And the platform’s identity design trades a network hop per request for immediate platform-wide revocation; at classroom scale we consider that trade obviously correct, and token introspection results are cacheable if it ever binds.

VIII. CONCLUSION AND FUTURE WORK

Code Guru treats novice support as one problem with four faces: seeing the mistake, teaching past it, keeping collaboration productive, and sustaining motivation. Three of those four components now carry measured evidence rather than architecture alone: Code Coach’s per-site scoring removes a false-positive class whole-file judgment cannot see; PairPath’s tuned classifier separates five collaboration states well enough to beat a rule-based baseline by wide margins; and Study Guider’s struggle classifier and micro-lesson pipeline show precise triggering and architectural viability, respectively. The methodology behind Code Coach’s results — generation-verified training data, a human-written holdout, calibrated thresholds, and measurement before every design commitment — is the part of this work we set out to reuse across the platform, and PairPath’s evaluation shows that reuse holding.

Immediate next steps are the controlled user study spanning all measured components, an independent, non-developer pilot of Study Guider’s micro-lessons, extension of candidate-level gating to the remaining gated types, evaluation of Study Guider’s mastery-verification stage against real longitudinal data, completion of the gamification engine, and public distribution of the platform through the VS Code Marketplace backed by the Cloud Run deployment.

ACKNOWLEDGMENT

The authors thank the Department of Information Technology at SLIIT for facilities and support.

REFERENCES

- [1] U. Z. Ahmed, P. Kumar, A. Karkare, P. Kar, and S. Gulwani, “Compilation error repair: for the student programs, from the student programs,” in *Proc. 40th Int. Conf. Software Engineering: Software Engineering Education and Training (ICSE-SEET)*, Gothenburg, Sweden, 2018, pp. 78–87.
- [2] P. Denny, S. MacNeil, J. Savelka, L. Porter, and A. Luxton-Reilly, “Desirable characteristics for AI teaching assistants in programming education,” in *Proc. Innovation and Technology in Computer Science Education (ITiCSE)*, Milan, Italy, 2024, pp. 408–414.
- [3] R. Singh, S. Gulwani, and A. Solar-Lezama, “Automated feedback generation for introductory programming assignments,” in *Proc. 34th ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, Seattle, WA, USA, 2013, pp. 15–26.
- [4] T. Blok and A. Fehnker, “Automated program analysis for novice programmers,” in *Proc. 3rd Int. Conf. Higher Education Advances (HEAd)*, Valencia, Spain, 2017, pp. 1138–1146.
- [5] H. Park et al., “CodeDive: a web-based IDE with real-time code activity monitoring for programming education,” *Applied Sciences*, vol. 15, no. 19, Art. no. 10403, 2025.
- [6] H. B. Hassan, Q. I. Sarhan, and Á. Beszédes, “Evaluating Python static code analysis tools using FAIR principles,” *IEEE Access*, vol. 12, pp. 173647–173659, 2024.
- [7] M. M. Rahman, Y. Watanobe, and K. Nakamura, “A neural network based intelligent support model for program code completion,” *Scientific Programming*, vol. 2020, Art. no. 7426461, 2020.
- [8] M. Kumar and M. K. Shah, “AIRA: AI-powered code review & bug detection system,” *Int. J. Scientific Research in Engineering and Management*, vol. 9, no. 3, 2025.
- [9] P. Preethi, V. Ragavan, C. Abinandhana, G. Umamaheswari, and D. R. Suvethaa, “Towards CodeBlizz: developing an AI-driven IDE plugin for real-time code suggestions, debugging, and learning assistance,” in *Proc. Int. Conf. Emerging Research in Computational Science (ICERCS)*, 2024.
- [10] H. A. Alrakhawi, N. Jamiat, and S. S. Abu-Naser, “Intelligent tutoring systems in education: a systematic review of usage, tools, effects and evaluation,” *J. Theoretical and Applied Information Technology*, vol. 101, no. 4, pp. 1048–1064, 2023.
- [11] N. A. Anindya Putri, R. A. Yuana, and P. Hatta, “Enhancing students’ ability in learning process of programming language using adaptive learning systems: a literature review,” *Open Engineering*, vol. 10, no. 1, pp. 820–829, 2020.
- [12] H.-C. Ling and H.-S. Chiang, “Learning performance in adaptive learning systems: a case study of web programming learning recommendations,” *Frontiers in Psychology*, vol. 13, Art. no. 770637, 2022.
- [13] H. Choi, H. Lee, and M. Lee, “Optimal knowledge component extracting model for knowledge-concept graph completion in education,” *IEEE Access*, vol. 11, pp. 15302–15315, 2023.
- [14] M. Shaka, D. Carraro, and K. N. Brown, “Personalised programming education with knowledge tracing,” in *Proc. Conf. Human Centered Artificial Intelligence: Education and Practice*, Dublin, Ireland, 2023.
- [15] E. Fotopoulou et al., “Expert agents for social and emotional learning: an agentic graph retrieval augmented generation approach,” *IEEE Access*, vol. 14, pp. 37568–37582, 2026.
- [16] L. Williams and R. Kessler, “Pair programming: more learning and less anxiety in a first programming course,” in *Proc. SIGSE Technical Symposium*, 2000, pp. 33–37.
- [17] J. Tan, L. Wu, and S. Ma, “Collaborative dialogue patterns of pair programming and their impact on programming self-efficacy and coding performance,” *British J. Educational Technology*, vol. 55, pp. 1060–1081, 2024.
- [18] A. A. Alaboudi and S. A. Alharbi, “Pair programming for learning programming: a systematic literature review,” *IEEE Access*, 2024.
- [19] M. Bigman, E. Roy, J. Garcia, M. Suzara, K. Wang, and C. Piech, “PearProgram: a more fruitful approach to pair programming,” in *Proc. SIGSE Technical Symposium*, 2021.
- [20] G. Fan, D. Liu, R. Zhang, and L. Pan, “The impact of AI-assisted pair programming on student motivation, programming anxiety, collaborative learning, and programming performance,” *Int. J. STEM Education*, vol. 12, no. 16, 2025.